

Examen Parcial de GRAU-IA

(15 de noviembre de 2017)

grupo 10

Duración: 55 min

1. (6 puntos) Una de las labores de los equipos de comunicaciones en una red es distribuir el flujo de paquetes que le llegan a un equipo entre los diferentes equipos con los que está conectado, optimizando la calidad de la distribución de paquetes de forma local.

Una posible estrategia de distribución de paquetes sería decidir a priori cuantos de los paquetes que se reciben en cierto segundo se envían por cada conexión, sin preocuparnos exactamente de a donde deben ir.

Un equipo de comunicaciones recibe cierto número de paquetes por segundo (P) que tiene que distribuir entre sus conexiones de salida. Para cada una de las N conexiones de salida del equipo se conocen tres informaciones: su capacidad (en número de paquetes por segundo), el tiempo de retraso que introduce la conexión (tiempo medio adicional que añade el nodo de salida al tiempo de llegada a destino de cada paquete) y el número medio de saltos que hará cada paquete hasta llegar a su destino.

Deseamos calcular el número de paquetes que debemos enviar por cada conexión de salida para optimizar la distribución de paquetes de manera que el retraso y número medio de saltos de los paquetes sea el mínimo posible. Evidentemente se han de enviar todos los paquetes que llegan.

En los siguientes apartados se proponen diferentes alternativas para algunos de los elementos necesarios para plantear la búsqueda (solución inicial, operadores, función heurística, ...). El objetivo es comentar la solución que se propone respecto a si es correcta, es eficiente, o es mejor o peor respecto a otras alternativas posibles. Justifica tu respuesta.

- a) Se plantea solucionarlo mediante Hill-Climbing usando como solución inicial el repartir todos los paquetes a partes iguales entre todas las conexiones de salida del equipo. Como operador utilizamos mover cierta cantidad de paquetes de una conexión a otra. Como función heurística usamos el sumatorio, para todas las conexiones con paquetes asignados, del producto entre el retraso de la conexión y el número de saltos.

Hill-Climbing es, a priori, una técnica adecuada para este problema en que se nos pide minimizar según unos criterios.

La solución es válida (es una asignación completa de paquetes a conexiones) siempre que el número medio de paquetes sea menor o igual que la capacidad de la conexión que admita menos paquetes, en caso contrario podría darnos una no-solución. El coste de calcular la solución es $O(\text{paquetes})$. Lo que podemos decir sobre la calidad de la solución inicial es que equilibra el retraso de todas las conexiones, no será la mejor solución inicial, pero tampoco será la peor.

El operador *mover* nos permite generar todas las soluciones posibles, ya que todos los paquetes están asignados desde la solución inicial, pero debería comprobar las restricciones del problema (no superar los límites de capacidad) para que no salga del espacio de soluciones. Al poder mover paquetes de cualquier conexión a cualquier otra y tener un parámetro extra para poder mover más o menos paquetes con el mismo operador el factor de ramificación sería $O(\text{conexiones}^2 \times \text{paquetes})$, lo que supone un factor muy grande.

La función heurística es aparentemente correcta ya que al minimizarla llegamos a soluciones mejores, el problema que tiene es que para distintas distribuciones de

paquetes dará el mismo valor, por lo que tiene muchas mesetas y no es capaz de distinguir bien entre soluciones mejores o peores. Sería mas adecuado incluir el número de paquetes por conexión en el heurístico de forma que el operador mover tuviera un efecto directo en el valor del heurístico. Y dado que la solución inicial o los operadores pueden salir del espacio de soluciones, el heurístico debería penalizar las no-soluciones.

- b) Se plantea solucionarlo mediante Hill-Climbing tomando como solución inicial el repartir los paquetes entre las conexiones de salida del equipo asignando aleatoriamente a una conexión el máximo de su capacidad, repitiendo el proceso hasta haber repartido todos los paquetes. Como operador se utilizaría mover cualquiera de los paquetes de una conexión a otra siempre que la operación sea válida. Como función heurística usamos el sumatorio para todas las conexiones con paquetes asignados del producto entre el número de paquetes asignados a la conexión y el retraso de la conexión más el sumatorio, para todas las conexiones con paquetes asignados, del producto entre el número de paquetes asignados a la conexión y el número de saltos medio de esa conexión.

De nuevo Hill-Climbing es una elección adecuada para este problema.

La solución inicial es correcta (es una asignación completa de paquetes a conexiones que no excede la capacidad de ninguna conexión), su coste es $O(\text{paquetes})$ pero no tenemos ninguna garantía sobre su calidad.

El operador es válido, cubre todo el espacio de soluciones y su factor de ramificación es $O(\text{conexiones} \times \text{paquetes})$. Teniendo en cuenta que todos los paquetes de una conexión son iguales solo tiene sentido hacer la operación una vez por conexión. Al ser un operador de granularidad tan pequeña hará también que la búsqueda sea mucho mas larga.

La función heurística tiene en cuenta todos los elementos del problema. Si la minimizamos obtendremos la solución que buscamos. No penaliza no-soluciones, pero en este caso no hace falta ya que no se generarán. El único defecto que tiene es que al estar mezclando dos criterios distintos las magnitudes que tienen pueden hacer que un criterio tenga preferencia respecto al otro a la hora de buscar una solución, y por ello lo mejor sería añadir un factor ponderante a cada sumando.

- c) Se plantea solucionarlo mediante Algoritmos Genéticos, donde la representación del problema es una tira de bits compuesta por la concatenación de la representación en binario del número de paquetes asignados a cada conexión (evidentemente usando el mismo número de bits para cada conexión). Como solución inicial ordenamos las conexiones por su retraso (de menor a mayor) y asignamos paquetes en ese orden llenando la capacidad de cada conexión hasta haber asignado todos los paquetes. Usamos los operadores de cruce y mutación habituales. Usamos como función heurística el sumatorio, para todas las conexiones, del producto entre el número de paquetes asignados a cada conexión y el retraso de la conexión.

Algoritmos Genéticos es también una técnica adecuada al tener que obtener soluciones que minimizan unos criterios.

La codificación permite representar correctamente todas las soluciones, pero también no-soluciones si el número de bits asociado a cada conexión puede representar números por encima de la capacidad de la misma.

La solución inicial es solución, su coste $O(\text{conexiones} \times \log(\text{conexiones}) + \text{paquetes})$ no será demasiado grande asumiendo que el número de conexiones no es demasiado alto.

Esta solución intenta optimizar uno de los criterios (el retraso) esto no significa que la solución inicial sea buena respecto al objetivo del problema, no tenemos garantías de estar cerca de la solución aunque probablemente sea mejor que las alternativas de los otros apartados. El problema es que solo podemos generar una solución.

Los operadores de cruce y mutación pueden dar lugar a soluciones invalidas ya que no se comprueba la capacidad de las conexiones, e incluso pueden alterar el número total de paquetes asignados, que debería ser siempre P .

La función heurística es incompleta ya que solo tenemos en cuenta el retraso de las conexiones, pero no el número de saltos, y no penaliza no-soluciones.

2. (4 puntos) En todo proyecto de desarrollo de una aplicación informática es necesario determinar cuanto tiempo y recursos va a ser necesario invertir. Una vez decididas las tareas, su duración y la dependencia entre ellas podemos determinar automáticamente cuando se van a desarrollar y qué personas van a hacer cada tarea.

Supondremos que hemos dividido el problema en tareas (t_1 a t_m) que tienen todas la misma duración y hemos determinado su grafo de dependencias, de manera que para cada una de ellas sabemos qué tareas han de estar terminadas para poder empezarla. Disponemos también de un conjunto de programadores ($prog_1$ a $prog_n$) que podemos asignar a cada una de las tareas. Obviamente si asignamos el mismo programador a dos tareas que pueden realizarse simultáneamente tardaremos más que si asignamos programadores diferentes, pero siempre nos saldrá más barato usar un número menor de programadores.

El objetivo del problema es encontrar la menor asignación de programadores a tareas que respete el orden de dependencia de estas.

- a) Queremos usar A^* definiendo el estado como la asignación de programadores a tareas. El estado inicial sería la asignación vacía y el estado final sería la asignación completa de programadores a tareas. Como operadores usaremos dos: asignar un programador cualquiera a una tarea cualquiera, el coste del operador sería una unidad de duración; y desasignarle a un programador una tarea, el coste de este operador sería una unidad negativa de duración. La función heurística es el número de tareas que quedan por asignar que tengan alguna tarea de la que dependan que no esté ya asignada.

El algoritmo A^* es adecuado en este caso, en el que se pide optimizar la asignación de programadores.

Es correcto definir el estado como la asignación de programadores a tareas. El problema se define como la búsqueda de un camino que se corresponde con la secuencia de asignaciones de programadores a tareas.

Tal como está planteado el operador de asignar programador, el número de asignaciones posibles a cada paso se corresponde con el producto entre programadores y tareas, teniendo un factor de ramificación de $O(m \times n)$. Por las características del problema se podría reducir este factor de ramificación si consideramos una ordenación de las asignaciones que respete el orden de precedencia de las tareas. De esta manera a cada paso solamente unas pocas tareas podrán ser asignadas. El coste del operador asignar programador es siempre 1 y tiene como consecuencia que todas las soluciones tengan el mismo coste, por lo que no se podrán distinguir las soluciones que usan más o menos programadores para completar las tareas.

El operador de desasignar programador no tiene sentido en A^* , ya que es el propio algoritmo el que va construyendo la solución desde cero y en vez de desasignar simplemente escoge otra rama más prometedora. Además A^* no puede tener operadores con coste negativo.

La función heurística es admisible ya que dará en todo momento un valor menor o igual que el número de tareas que hacen falta para completar la solución y, tal como está planteada, dará preferencia a los caminos que asignen programadores a las tareas que bloquean otras tareas. Esto en principio es positivo ya que hace que se asignen primero las tareas de las que dependen otras, el problema es que este heurístico pasa a valer cero cuando se han asignado esas tareas de las que otras dependen, aunque queden tareas por ejecutar. De hecho en casos en los que no haya dependencia entre tareas este heurístico valdría 0 ya en el estado inicial y no se asignarían programadores a ninguna tarea.

A pesar de que el coste g y el heurístico h están expresados en unidades diferentes (duración y tareas), al ser la duración siempre unitaria por cada tarea asignada en realidad el coste g está contando las tareas ya asignadas, por lo que la suma $f = g + h$ es correcta. Sin embargo, ni g ni h tienen en cuenta el número de programadores y por ello, tal y como está planteado, este A^* no puede optimizar el número de programadores que se asignan en las tareas, lo que hace que este planteamiento no resolvería el problema.

- b) Queremos usar Satisfacción de Restricciones, donde las variables son las tareas a asignar y los programadores los dominios de las variables. Las restricciones son las relaciones de dependencia entre las diferentes tareas, estableciendo que un programador no puede estar asignado a 2 tareas entre las que haya una dependencia. Sobre estas restricciones imponemos además que no pueda haber más de tres programadores diferentes en tareas que pueden realizarse en paralelo.

A priori es un error escoger Satisfacción de Restricciones para este problema, ya que es una técnica que no permite ni maximizar, ni minimizar, ni optimizar.

Es correcto definir las tareas como las variables y los programadores como los dominios ya que de esta forma la representación asegura que, al reducir los dominios de todas las variables a un solo valor, se representa una solución donde a todas las tareas se les asigna un programador y solo uno. El hacerlo al revés representaría tener que trabajar con los programadores como variables que tienen un conjunto de valores (las tareas), y al reducir los dominios de las variables a un solo valor tendríamos que cada programador puede tener solo una tarea asignada.

La primera restricción permite representar el grafo de precedencia de las tareas, pero se aplica de forma incorrecta. No tiene sentido que un programador no pueda ser asignado a dos tareas si una depende de la otra, en todo caso debería ser lo contrario.

La restricción adicional va encaminada a limitar el número de programadores que se usan para tareas que pueden realizarse simultáneamente, de manera que se hagan como máximo tres tareas en paralelo. Este número es arbitrario y no garantiza que la asignación que se encuentre sea mínima o que se garantice usar más de una persona para tareas simultáneas, y se añade para intentar asegurar una calidad mínima de la solución que encuentre el algoritmo.

Comenta cada una de las posibilidades indicando si resuelven o no el problema, qué errores te parece que tiene cada solución y cómo se podrían corregir, y qué ventajas e inconvenientes tienen cada una de ellas. Justifica la respuesta.